

 TEST — INDIVIDUAL

Test: Restaurant Waiter Order System

Build a simple CRUD Flutter app for a waiter to manage the restaurant menu and take customer orders. No login or registration — focus is on full CRUD and a working database connection.

**Deadline****Saturday, 18 July 2026 · 11:59 PM**Submit via **eLearning** (individual submission)

1. Introduction

Restaurants need a quick way to record customer orders and keep the menu up to date. This test asks you to build a small **Order Management app** used by a **waiter**. The waiter can add menu items, take orders for a specific table, and track each order through its lifecycle until it is paid.

The app is intentionally small so you can finish it within the test window — but it must demonstrate **complete CRUD** (Create, Read, Update, Delete) and a real connection to a **cloud database**.

**Who uses the app?**

- **Waiter** — only user. Adds menu items, takes orders, updates order status, marks as paid.
- No login, no registration, no role management — every device opens straight into the app.

2. Software, Language, Tools, API, Templates

Category	What to use
Language	Dart
Framework	Flutter (latest stable)
IDE	Visual Studio Code with the Flutter & Dart extensions

Flutter App Development Course

Category	What to use
Emulator	Android emulator from Android Studio (or a real Android device)
Backend (pick one)	Firebase (Firestore) — OR — Supabase (Postgres + RLS)
Backend SDK	firebase_core + cloud_firestore — OR — supabase_flutter
State management	setState (sufficient) — or Provider if you prefer
UI templates	Material 3 widgets — Scaffold, ListView, Card, TextFormField, AlertDialog, BottomNavigationBar
Version control	Git + GitHub repository (recommended)

Pick your backend track

You may use either backend — both are fully covered in this course. Follow the matching lesson series to set up your project, then come back here.



Firebase

FIRESTORE · EASIEST

Documents & collections, real-time snapshots. Follow Lessons 11a → 11e for setup and CRUD patterns.

Open Firebase lessons



Supabase

POSTGRES · SQL

Real SQL tables with RLS. Follow Lessons 11a-Supabase → 11e-Supabase for setup, schema, and CRUD.

Open Supabase lessons



3. Development Steps

Build the app in this order. Each step builds on the previous one:

1. **Create the Flutter project** in VS Code (`flutter create resto_order`) and run the default app to confirm the emulator works.
2. **Create the backend project** — a Firebase project (enable Firestore in test mode) OR a Supabase project (Singapore region for Malaysia).
3. **Add the SDK** — `flutter pub add cloud_firestore firebase_core` OR `flutter pub add supabase_flutter` , then initialise in `main()` before `runApp()` .

Flutter App Development Course

4. **Create the schema** — three collections/tables: `menu_items` , `orders` , `order_items` (see Data Model below). For Supabase, open public read/write policies are fine for this test.
 5. **Build the Menu tab** — list menu items, add new item form, edit, delete, and a toggle for `available` .
 6. **Build the Orders tab** — list orders grouped/filtered by status, plus a "New Order" screen where the waiter picks a table number and adds line items from the menu.
 7. **Build the Order Detail screen** — show items, total, and buttons to advance the status: `Pending` → `Preparing` → `Served` → `Paid` . Allow delete (cancel) while status is `Pending` .
 8. **Test every flow** on the emulator — add a menu item, take an order with 2–3 line items, advance the status to Paid, then delete a stale menu item.
 9. **Take screenshots** of every screen for your report.
 10. **Write the report** using the four sections required by this brief.
 11. **Zip the Flutter project** (exclude the `build/` folder) and submit on eLearning together with the PDF report.
-

Data Model

The app uses **three** collections/tables. `order_items` is the link between an order and the menu items it contains — one order can have many items.

**DATA MODEL — 3 TABLES / COLLECTIONS****menu_items****PK** id

name

price

double

category

e.g. Drink

available

bool

**orders****PK** id

table_no

status

enum

total

double

created_at

**order_items****PK** id**FK** order_id**FK** menu_item_id

name_snapshot

price_snapshot

quantity



orders 1 → many order_items · order_items.menu_item_id → menu_items.id

Flutter App Development Course

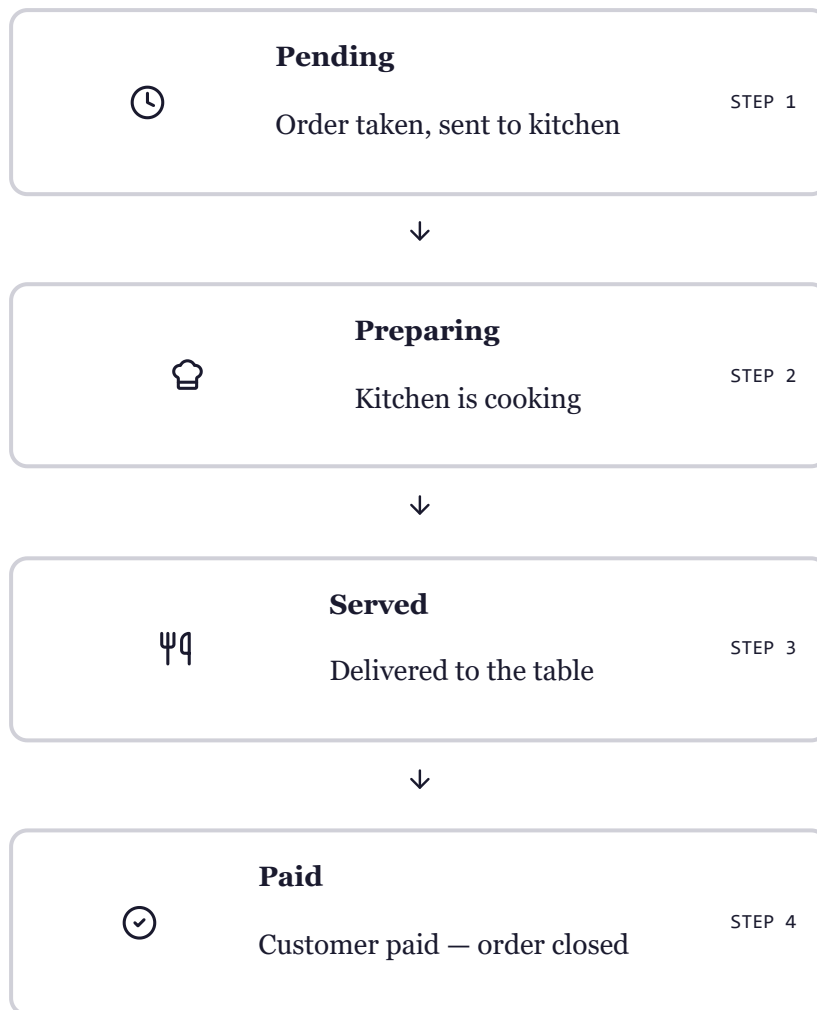
Use `name_snapshot` & `price_snapshot` so historical orders stay correct even if a menu item is later edited or deleted.

Why snapshots? Copy the menu item's name and price into the `order_items` row at the time the order is created. That way, if the price of a menu item changes later, old orders still show what the customer actually paid.

Order Lifecycle

Every order moves through four statuses, in order. The waiter taps a button on the order detail screen to advance the status. While the order is still **Pending**, it can be edited or cancelled (deleted).

ORDER LIFECYCLE (STATUS ENUM)



While status is **Pending**, the order can still be edited or deleted (cancelled).

Each transition is triggered by a button in the order detail screen.

App Flow

Keep the app to five screens total. Use a bottom tab bar (Menu / Orders) for the two top-level sections, then push detail and form screens with `Navigator.push`.

WAITER APP — SCREEN FLOW

Home (tabs)
 Menu | Orders

MENU TAB

→

Menu list



edit / delete / toggle available

→

Add menu item

+

name, price, category

→



Edit menu item

ORDERS TAB

→

Order list



filter by status

→

New order

+

pick table + items + qty

→

Order detail



advance status / cancel

Only 5 screens total — keep navigation simple with bottom tabs + push routes.

CRUD coverage required

- **Menu** — Create, Read (list), Update, Delete, toggle `available` .
- **Orders** — Create (with line items), Read (filter by status), Update (advance status), Delete (cancel while Pending).

Flutter App Development Course

- **Order items** — Created with the parent order. Quantity editable while order is `Pending` .

4. System Interface (Required Screenshots)

Your report's Section 4 must include **one screenshot per screen** below, taken from the emulator. Add a short caption (1–2 sentences) under each image describing what the screen does.

#	Screen	Must show
1	Menu list	List of menu items with name, price, available toggle, edit/delete icons
2	Add / Edit menu item	Form with name, price, category fields and Save button
3	Orders list	Orders grouped or filtered by status (Pending, Preparing, Served, Paid)
4	New order	Table number input + add-item selector with quantities + total
5	Order detail	Items list, total, and the button to advance the status



Tip — emulator screenshots

In the Android emulator's side toolbar, click the **camera icon** to capture the current screen. Files are saved to your Desktop on macOS / Pictures on Windows.

Submission Instructions



What to submit on eLearning

1. **Flutter project folder** — zipped, excluding the `build/` folder.
2. **Report PDF** — covering all 4 required sections above.
3. Upload to **eLearning** with file name: `FinalTest_<MatricNo>_<Name>.zip`

**Report structure (PDF)**

- **Cover page** — name, matric number, section, lecturer name, date.
- **Section 1** — Introduction.
- **Section 2** — Software, language, tools, API, templates used.
- **Section 3** — Development steps (with short notes on what you did at each step).
- **Section 4** — System interface (the 5 captioned screenshots).

**Deadline**

Saturday, 18 July 2026 · 11:59 PM

Submit on **eLearning**. Late submissions follow the standard course penalty policy.

Marking Rubric (100 marks)

Area	Marks
Report quality (Sections 1–4)	20
Menu CRUD working (add/edit/delete/list)	20
Order CRUD + line items (order_items)	25
Status lifecycle transitions (Pending → Paid)	15
Backend integration (Firebase or Supabase)	10
UI/UX & code quality	10
Total	100

Important Reminders



No login or registration required

Do **not** add authentication. The app opens straight into the Home tabs. Spending time on login takes marks away from the CRUD areas that are graded.



The database must be a real cloud backend

Saving with `shared_preferences` or in-memory lists is **not acceptable**. Your data must persist in Firestore or Supabase Postgres so it survives an app restart and works across devices.



Keep the scope small

Do not add features outside this brief (no reporting, no charts, no printing, no multi-restaurant support). The grader checks the CRUD coverage and the order lifecycle — finish those first.

Quick Summary



Submission checklist

- Flutter app runs on the Android emulator with no crashes
- 3 tables/collections created (menu_items, orders, order_items)
- Menu CRUD complete (add, edit, delete, toggle available)
- Order CRUD complete (create with line items, list, advance status, cancel)
- Status flow works: Pending → Preparing → Served → Paid
- 5 screen screenshots taken for the report
- Report PDF covers Sections 1–4
- Zip + PDF uploaded to eLearning before 18 July 2026, 11:59 PM

Flutter App Development Course

Need a refresher?

Review CRUD patterns and backend setup before you start.

[Lesson 3 — CRUD](#)[Firebase track](#)[Supabase track](#)